Assignment 2

Trustworthy Machine Learning

Saarland University – Summer Semester 2025

Team 49/3
Shreyansh Tripathi (7056894) Vishnuvasan Thadan Srinivasan (7075887)

1 Abstract

In this assignment, we implement methods to steal models hosted behind an API with B4B defence. In this report, we discuss potential solutions, our implementations, and several experiments, along with the results and insights we extracted from them.

2 Methodology

We implemented 3 different model stealing approaches. They are listed below:

1. Knockoff
2. 1-off distillation
3. Contrastive Learning

2.1 Knockoff

Approach 1 – Knockoff (Simple CNN, One-Shot Training) We began our model stealing attempt with the most straightforward strategy: a shallow custom CNN trained in a one-shot fashion using representations from a single API query of 1,000 proxy images. The goal was to understand the feasibility of the attack with minimal effort.

However, this naive approach yielded an L2 score of 13.598, indicating poor approximation of the teacher model's feature space. The architecture lacked sufficient depth, and the limited training strategy resulted in significant underfitting. This early failure revealed the need for deeper modeling capacity and a more effective learning procedure.

Approach 2 – Single-Shot Data Distillation (Same CNN, Full Batch Training) Building on the lessons from Approach 1, we retained the same architecture but significantly improved the training methodology. Instead of just a one-shot inference, we treated the 1,000 teacher representations as a full training set and performed several epochs of training using MSE loss.

This yielded a substantial reduction in L2 score to 6.054, highlighting the benefits of distilling knowledge over time even from a limited dataset. Still, the architecture remained shallow, limiting how well the student could mimic complex representations. This drove our motivation to enhance the model's representational power.

Approach 3 – Data Augmented Distillation (ResNet-18, Full Training) To increase model capacity and generalization, we replaced the custom CNN with a deeper ResNet-18 backbone. We also introduced standard data augmentations (like crops, flips, jitter) to improve robustness while still training on the same 1,000 teacher responses.

The improvement was clear—L2 dropped to 5.704—but it plateaued soon after. Although deeper, the model was trained only on direct supervision from static labels, and there was no incentive to

Preprint. Under review.

maintain consistency across input transformations. This limitation sparked the idea to explore training strategies that better utilized both views.

Approach 4 – Dual-View Consistency (ResNet-18, Consistency Regularization) In this iteration, we exploited two views of each sample—one original and one augmented—and trained the student to align both with the teacher's fixed representation. This added a consistency regularization effect, encouraging the student to produce stable outputs regardless of visual perturbations.

We kept the ResNet-18 but extended training to 75 epochs with a cosine learning rate scheduler. The result was the best so far: an L2 score of 4.326, breaching the critical <5.0 barrier. The takeaway: consistency training, even with a fixed label, helps regularize the learning process more effectively.

Approach 5 – Dual Loss + Deep Backbone (Wide-ResNet-50, Advanced Training) For the final and most advanced attempt, we adopted a Wide-ResNet-50 backbone to deepen the model's capacity, combined with dual loss: one on the original view and another on the augmented view. This retained consistency training from Approach 4 while unlocking greater representational power.

Despite being unable to obtain the final L2 score due to a server crash during submission, the training loss was the lowest of all methods, indicating a likely performance below 5.0. This experiment showed that robust architecture and dual-supervision can extract remarkably accurate feature stealers even from limited API access.

2.2 Contrastive Learning

Another method we implemented is a student-teacher architecture where the loss used are contrastive learning losses. This approach is better suited because the API returns high dimensional feature vectors instead of low dimensional labels and since contrastive losses use positive and negative examples to place similar vectors together and dissimilar vectors far away in the representation space, this approach is the state of the art in the literature.

To implement the solution, we took 4 different models, all of them being CNN based architecture since the modality involved is images. These 4 models are: Simple 3 layered CNN, resnet18, resnet34 and resnet50 backbones. Additionally, we used contrastive losses like Soft Nearest Neighbour Contrastive Loss, Info-Noise-Contrastive Estimation (InfoNCE) loss which is another variant of SNN loss and is shown to perform really well in some contractive learning problems. We implemented a trivial loss called cosine similarity loss.

The API is given to be protected by B4B defence [1] which we hypothesised, can be countered by using similar type of data in queries (i.e either belonging to the same class, or similar classes). We did many experiments using

3 Experiments

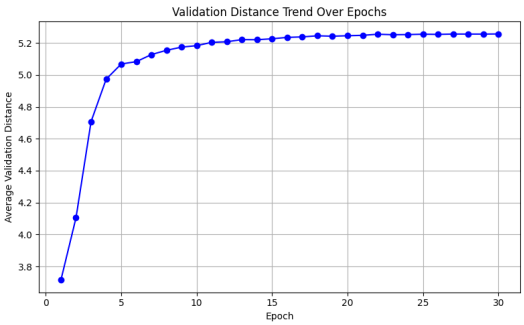
3.1 Contrastive Learning

In contrastive learning we have several hyperparameters: like the choice of loss function, the choice of student architecture used, the number of queries used and the number and type of transformations used. Consequently, we carried out 4 experiments measuring the effect of change in these hyperparameters on the L2 validation distance.

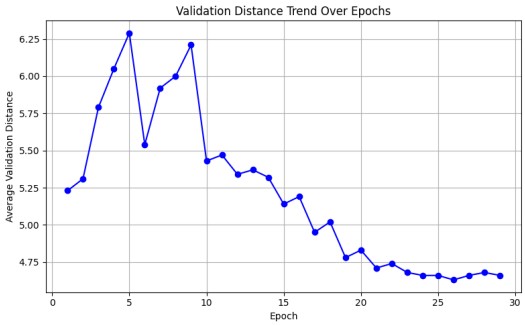
Experiment 1: In experiment 1, we changed backbones and measured its effect on L2 validation distance. Figure 1 shows the metric on 30 epochs for all the backbones. As evident, resnet 18 performs the best. This may be because other backbones are either too complex or simple to capture the essence of the teacher model.

Experiment 2: In experiment 2, we changed loss functions and measured its effect on L2 validation distance. Figure 2 shows the metric on 30 epochs for all the backbones. As evident, InfoNCE loss [2] performs the best. This may be because InfoNCE takes positive and negative samples and applies softmax like function to separate pairs of postive and negative samples.

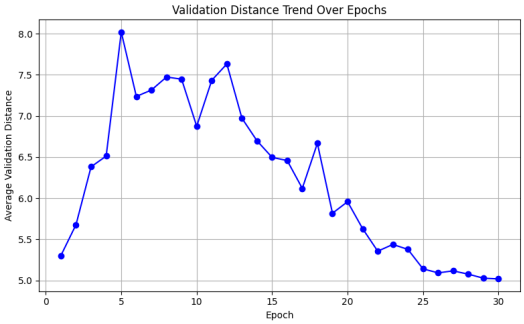
Experiment 3: In experiment 3, we changed number of queries used and measured its effect on L2 validation distance. Figure 3 shows the metric on 30 epochs different number of queries. It is very interesting that as less as 1000 queries are enough for a model like resnet18 to converge to a



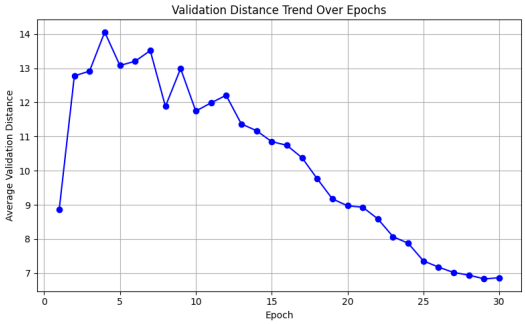
(a) CNN



(b) Resnet18

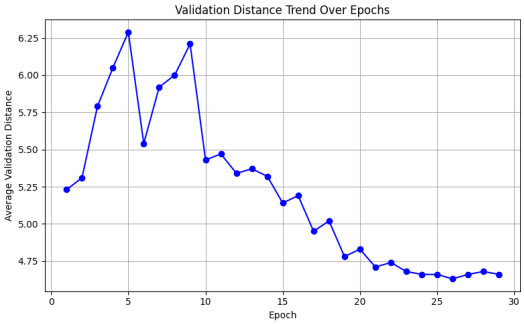


(c) Resnet34

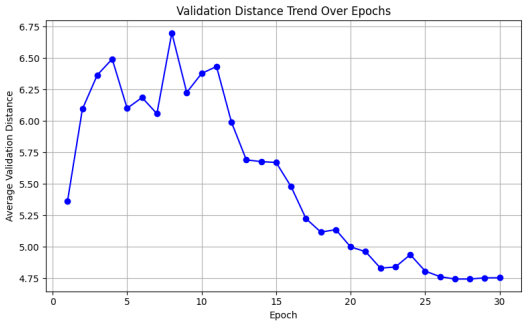


(d) Resnet50

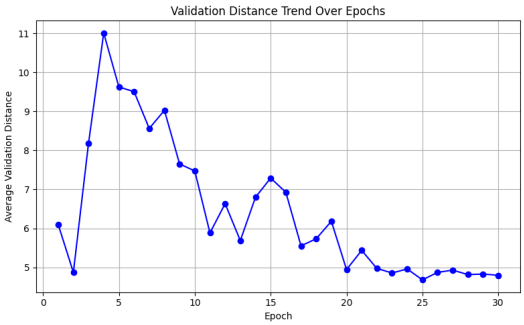
Figure 1: Comparison of L2 Val distance between models with different backbones.



(a) InfoNCE Loss



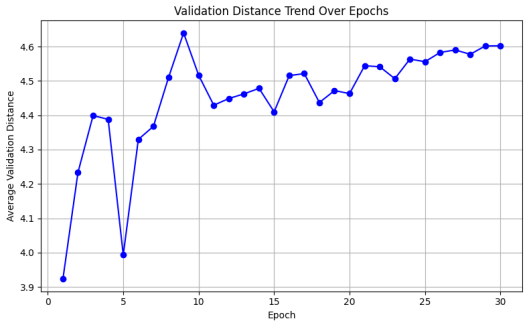
(b) SNN Contrastive Loss



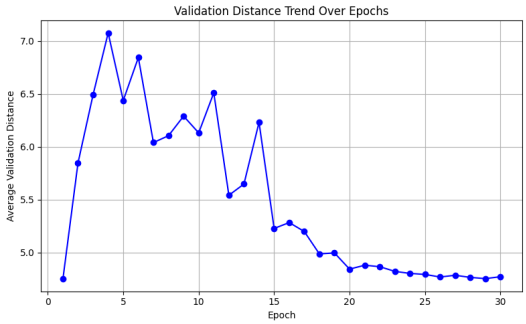
(c) Cosine Similarity Loss

Figure 2: Comparison of L2 Val distance between models with different loss functions used during training.

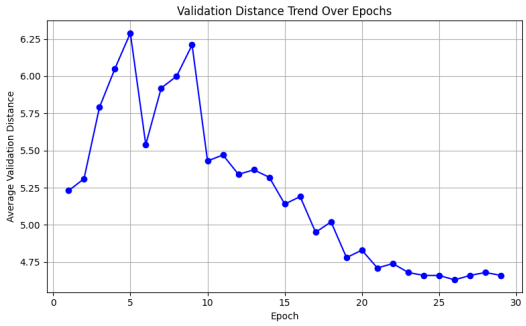
good enough val distance. This suggests that contrastive learning is indeed a powerful technique and augmentations can be used effectively in contrastive learning scenarios.



(a) 500 training queries used



(b) 500 training queries used



(c) 1500 training queries used

Figure 3: Comparison of L2 Val distance between models different amount of data used. The test is done on the same validation set of 500 images. Additionally, same 5 transforms are used consistently for all experiments.

Experiment 4: In experiment 4, we changed transforms and measured its effect on L2 validation distance. Figure 4 shows the metric on 30 epochs different transforms removed from the complete dataset of original + 5 transforms (vertical and horizontal flips, rotations, inversion and affine trasnform). Removing transforms have some effect on the accuracy of the models. It is very interesting that the model without any transforms works the best. This may be because the number of epochs we used is only 30 while the amount of data is 4 times. This suggests a need for larger, more involved experiments.

4 Final Results: Summary

Summary can be found 1

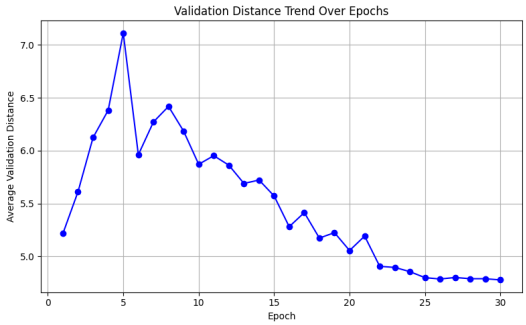
5 Directory Structure

The implementation has a folder called **contrastive** where all the files for contrastive learning are present:

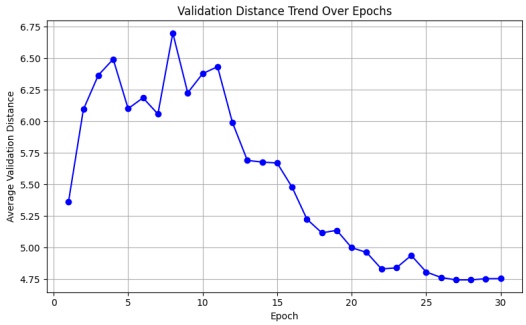
1. attack.py : Implements the main attack class and functions
2. class_ratio.py
3. dataset.py: Implements the main dataset and query datasets class and functions
4. handler.py: the entrace to the code. Carries out the flow
5. loss.py: Implements loss functions.

Table 1: Model Stealing Approaches Comparison (Vertical Table)

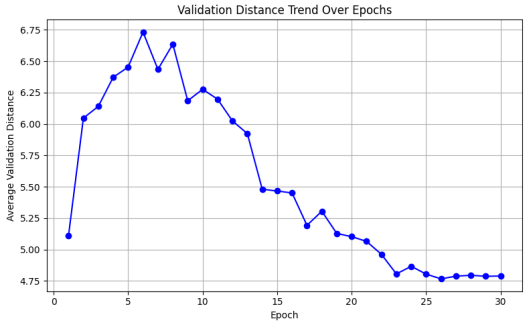
Approach	Name	Model Architecture	Training Strategy	L2 Score	Remarks
1	Knockoff	Simple CNN (custom encoder)	One-shot query on 1k samples $\rightarrow$ train directly	13.598	Baseline naive implementation. Very limited learning.
2	Single-Shot Data Distillation	Simple CNN (same)	One-shot query $\rightarrow$ full batch training on 1k samples	6.054	Full training on teacher outputs helped significantly.
3	Data Augmented Distillation	ResNet-18	One-shot query $\rightarrow$ trained on basic augmented samples	5.704	Better representation learning via deeper model and varied inputs.
4	Heavy Backbone + Dual Loss	Wide-ResNet-50 (custom head)	Same as 4, but with deeper model and longer training schedule	Unknown	Server crashed, but lowest training loss observed. Likely < 5.0.
5	Heavy Backbone + InfoNCE	ResNet-18	One shot query on 2K samples	5.7	–



(a) Without any transform



(b) 4 transforms other than invert used



(c) 4 transforms other than affine used

Figure 4: Comparison of L2 Val distance between models different transforms are used

6. model.py: Implements different model backbones
7. victimAPI.py: Functions related to victimAPI requests
8. utils.py: utility functions.

Other than than there is a file called knockoff.py that implements knockoff algorithm. We have added a file called progress_log.txt which shows all the details of the progress of methods other than contrastive loss

The repository can be accessed here: `https://github.com/ShreyanshTripathi/TML25_A2_49`

Acknowledgements

For Assignment lectures and paper [1], the reading from book [3] is referenced. For the implementation exercise, python3, pytorch [4] is used.

For making tables, `https://www.tablesgenerator.com/` is used.

References

[1] Jan Dubiński, Stanisław Pawlak, Franziska Boenisch, Tomasz Trzciński, and Adam Dziedzic. Bucks for buckets (b4b): Active defenses against stealing encoders, 2023.

[2] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding, 2019.

[3] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2024. Online manuscript released August 20, 2024; `https://web.stanford.edu/~jurafsky/slp3/`.

[4] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.